

Zero-Overhead Abstraction

Moving Cost Out of Runtime — Compile-Time Polymorphism, Compile-Time Computation, and Preallocation

Robert (Bo) Hou · Robert's Technical Notes · June 2026

Abstract. C++'s founding promise is a single sentence: “**What you don’t use, you don’t pay for; what you do use, you couldn’t hand-code any better.**” This is *zero-overhead abstraction*. This report tests that promise across three themes — who *violates* it and how it is *restored*. **Virtual functions** violate it (runtime polymorphism blocks inlining); **CRTP** restores it with compile-time polymorphism. Old-style **template metaprogramming** can compute at compile time but is syntactically painful; **constexpr** restores it with ordinary function syntax. **malloc** is costly on the hot path; **arena/pool allocators** use preallocation to reduce allocation to a single pointer operation. One thread runs through all three: **move cost out of runtime** — into compile time (CRTP, constexpr) or into startup (preallocation). This is where the first two mountains (correctness, speed) land on the “code abstraction” side: data must be contiguous, predictable, reusable; abstraction must evaporate at compile time or startup, leaving no runtime cost on the hot path.

0. Zero-Overhead Abstraction: The Promise

At the core of Stroustrup’s definition of C++ is one promise, worth pinning here because all three parts of this report test it:

Zero-overhead abstraction. First, what you don’t use, you don’t pay for. Second, what you do use, you couldn’t hand-code any better. This promise is C++’s standing relative to higher-level languages: you get high-level abstraction (interfaces, generics, containers) without a runtime tax for it. This report examines, one by one, where virtual functions *violate* the promise, and how CRTP / constexpr / preallocated allocators *restore* it.

1. Virtual vs CRTP: Compile-Time Polymorphism Restores Inlining

A base interface, a derived implementation, called hundreds of millions of times in a hot loop. First, clear a **common misconception**: many say “virtuals are slow because of a cache miss each call” — **wrong**. In a hot loop the object, the vtable pointer, the vtable, and the target address are all accessed repeatedly and, after the first time, all hit in cache — **no per-call miss**. Likewise the indirect jump `call [reg]` can be learned by the indirect branch

predictor: as long as the type `p` points to is **stable**, the predictor learns it once and keeps guessing right — **the jump misses at most a few times**.

The real cost of a virtual: it blocks inlining, not the jump or the cache. A virtual call does three things over a plain call: read the object's vtable pointer, read the target address from the vtable, and **jump indirectly**. The first two hit in cache (cheap); the indirect jump, with a stable type, is nearly free. The cost that truly persists: **the compiler does not know `p`'s concrete type at compile time, so it dares not inline**. For a small hot function (say `return x*2+1;`) the body is smaller than one full function call; inlined it is nearly free and can be vectorized/constant-folded with the loop, un-inlined it pays the full call cost every time **and loses all cross-function optimization**. The cost is not in the jump or the cache — it is in “what should have been inlined wasn't.”

CRTP (the Curiously Recurring Template Pattern) exists to restore inlining. Make the base a template with the derived type as a parameter; the base then knows the concrete type at compile time and can inline:

```
template <class Derived>
struct Base {
    int compute(int x) const {
        return static_cast<const Derived*>(this)->compute_impl(x);
    }
    // Derived is a template param -> known at compile time -> inlinable
};
struct Impl : Base<Impl> { // the curious part: the class passes itself to its base template
    int compute_impl(int x) const { return x * 2 + 1; }
};
```

Why no infinite recursion? `struct Impl : Base` looks like “to define `Impl` you first need `Impl`,” but instantiating `Base` needs only `Impl`'s **name** (an incomplete type, since the base uses it only through a `Derived*` pointer, and a pointer needs no complete definition), while the base's member function bodies are **instantiated lazily, at call time** (by which point `Impl` is complete). This is the same as `struct Node { Node* next; };` — self-referential yet not recursive, because **the pointer breaks the recursion**. Only putting a `Derived` member (a complete object, not a pointer) in the base would truly fail.

Measured: the cost is not “that one jump,” but preserving the optimizer’s view.

For the small hot function $x*2+1$ called hundreds of millions of times, virtual vs CRTP: in a construction that cannot be optimized away wholesale (e.g. a dependency chain), virtual vs CRTP $\approx 1.27\times$ — the direct cost of “indirect jump + un-inlinable call.” In a vectorizable / wholly-eliminable construction, CRTP/direct get the **entire loop optimized away** (the compiler sees through the accumulation and uses a closed form or vectorizes), while the virtual must grind out hundreds of millions of indirect calls — the ratio tends to **infinity**. That is the real lesson: **a virtual’s cost is not the one jump saved, but that it seals off the compiler’s inlining and all cross-function optimization; once inlining is possible, what the compiler can do is unbounded.** (Methodology caveat: micro-benchmarks are easily contaminated — dependency chains, instruction scheduling, being optimized away all make numbers lie; construct with care.)

Trade-off: CRTP trades runtime flexibility for zero overhead and inlinability. It loses runtime polymorphism — a Base and a Base are **different types** and cannot go in one container. So CRTP fits hot paths where the type is known at compile time; where the type is decided at runtime and mixed in one container, virtuals still apply.

2. constexpr: Moving Computation to Compile Time

CRTP fixes the *type* at compile time; constexpr completes the *computation* at compile time. Historically “compile-time computation” had two hard roads: **macros** (preprocessor text substitution, ignorant of types/semantics, with the preprocessor **forbidding a macro from recursing on itself**, plus the classic traps of precedence, multiple argument evaluation, and semicolon swallowing) and **template metaprogramming** (recursive template instantiation simulating computation — **Turing-complete**; in 1994 Unruh computed primes at compile time with it — but syntactically brutal, encoding values as types, slow to compile, explosive error messages). constexpr (since C++11) replaces both with **ordinary function syntax**.

The core property of constexpr: one body, usable at compile time or runtime.

`constexpr int fib(int n){ if(n<2) return n; return fib(n-1)+fib(n-2); }` — this one function: when its result is required to be constant (e.g. `constexpr int a = fib(40);`) it is **forced to compute at compile time**, and at runtime is simply the constant 102334155 at zero cost; when the argument is a runtime value (e.g. `int b = fib(m);` with `m` from input) it **degrades to an ordinary runtime function**. **One body covers both execution times, per context** — precisely constexpr’s fundamental edge over template metaprogramming, which is **compile-time only** (`Fib` requires `m` to be a compile-time constant; you cannot write a runtime version).

Other advantages: constexpr uses normal `if/for/while` and locals, readable and debuggable, unlike metaprogramming’s specialization-as-condition and recursion-as-loop; and a **value is a value**, not encoded into a swarm of types, so it compiles faster with normal errors. Its power grows per standard: C++14 opens up function bodies, C++17 adds `constexpr` (compile-time branch), C++20 even allows `new`, `virtuals`, and `std::vector` in `constexpr`. Today most compile-time computation uses `constexpr`, with metaprogramming retreating to genuinely type-level work.

Three `const*` keywords (a frequent interview point). **`constexpr`**: *may* compute at compile time (when the context requires and arguments are constant) but is **allowed** to fall back to runtime. **`constexpr`** (C++20): *must* compute at compile time — passing a runtime variable is a compile error (the “forced compile-time” version). **`constexpr`** (C++20): guarantees a variable is *initialized* at compile time (fixing static-initialization-order problems), though the variable itself need not be constant. In one line: `constexpr` = can-be-compile-time (may fall back); `constexpr` = must-be-compile-time; `constexpr` = guaranteed compile-time initialization.

3. Custom Allocators: No new on the Hot Path

This part lands the conclusion of the Mechanical Sympathy mountain: no new on the hot path. First, where `malloc` is slow.

The cost of malloc, and its indirect harm. One malloc must: **traverse** a free list (usually a **doubly-linked list**) to find a block of suitable size; **bookkeep** (each block's header records size; on free, coalesce adjacent free blocks against fragmentation); **lock** under multithreading (the heap is shared); and, when the heap is short, ask the OS for memory via an mmap/sbrk **system call** (trapping into the kernel, very slow). "Taking a slot from a preallocated array" is just moving a pointer — tens to hundreds of times cheaper. **Indirect harm:** new'd objects are **scattered across the heap**, so traversing them later (especially a linked list) is pointer chasing → cache miss + failed prefetch (see Mechanical Sympathy). malloc is slow not only to allocate but in the *later access* of what it allocated.

3.1 Arena (bump / linear allocator)

Preallocate one large contiguous block with malloc, keep a pointer top to "the next free position." **Allocate** = `top += size`; return `old_top` — one pointer addition, $O(1)$, no traversal, no lock, no syscall. Object sizes may vary (all bumped forward). **Cost: no individual free** — top only moves forward; reclamation is a whole-block reset to the start. That is exactly why it is fast: giving up "individual free" removes the free list and bookkeeping, reducing allocation to an addition. Fits "a batch of objects with the same lifetime, dropped together" (e.g. reset the whole arena after a request/frame).

3.2 Pool (object pool / fixed-size allocator)

For **frequent allocation/free of same-size** objects (e.g. order nodes), use a pool: preallocate a large block cut into **fixed-size slots**. Since all slots are equal, slot *i*'s address = `base + i*slot_size`, **directly computable, regular addressing**; free slots are threaded on a free list, allocate pops the head, free pushes to the head, **both $O(1)$** — because all slots are equal, **any one works: no "finding a fit," no coalescing**, which is why it beats malloc's free list (malloc must handle arbitrary sizes, so it must search and coalesce).

Arena vs Pool: two trade-offs of the same preallocation idea. Shared: preallocate one large **contiguous** block (cache-friendly, prefetchable), $O(1)$ allocate, no malloc on the hot path. Difference: **arena** treats the contiguous space as "a tape you push forward," *any size*, bump-allocated, *no individual free* (whole reset); **pool** cuts it into *equal cells*, *regular addressing* (`base+i*size`), *individually freeable and reusable* (any free slot is equivalent). In short: pool trades "fixed size" for "individual free + regular addressing + brainless reuse"; arena trades "no individual free" for "any size + extreme simplicity (just a bump)." HFT uses both: a batch of same-lifetime temporaries dropped together → arena; frequently new/delete'd same-type objects → pool.

3.3 PMR: the standard-library landing (C++17)

C++17's `std::pmr` (polymorphic memory resource) lets standard containers **swap their allocator** without hand-writing one: `std::pmr::monotonic_buffer_resource` is the library's arena, `std::pmr::unsynchronized_pool_resource` its pool. Writing `std::pmr::vector`

`v{&arena;};` makes that vector allocate from your arena and **never touch malloc**. A frequent interview question — “how do you make `std::vector` avoid `malloc`?” — PMR + `monotonic_buffer_resource`.

4. Synthesis: Three Parts, One Thread; Three Mountains, One Conclusion

The three parts of zero-overhead abstraction share one theme — **move cost out of runtime**: CRTP moves “type resolution” to compile time (restoring inlining), `constexpr` moves “computation” to compile time (a free result at runtime), arena/pool move “allocation” to startup (leaving only a pointer op on the hot path). And these converge with the first two mountains at one point:

The shared engineering conclusion of three mountains. Lock-free (correctness) concluded: reuse array slots, never delete, sidestep safe reclamation, be immune to ABA. **Mechanical Sympathy (speed)** concluded: array addresses are predictable, so prefetch/MLP scale the memory wall; a list’s pointer chasing exposes latency. **Zero-overhead abstraction (this report)** concluded: preallocation + $O(1)$ allocation keeps `malloc` off the hot path, contiguous layout keeps objects cache-friendly, and CRTP/`constexpr` make abstraction evaporate at compile time. Three lines, one sentence: **if it can be contiguous, predictable, and reusable, do not dynamically allocate, do not chase pointers, and do not leave a cost at runtime that could have been settled at compile time or startup.** That is the underlying worldview of high-performance C++ — abstraction is free, so long as you move the cost to the right place.